

Autonomous Code Remediation Agent

Architecture & 2.5-Month Portfolio Roadmap

Project Vision

An open-source agentic system that monitors application logs for recurring failure patterns, traces them to root causes in a GitHub repository, autonomously generates fixes, and creates pull requests — with sensitive data handled by local models and complex reasoning handled by Claude.

Test Bed: [Haystack by deepset](#) — production-grade LLM orchestration framework used heavily in AI-first engineering teams worldwide.

Why Haystack? Complex Python codebase, rich GitHub issue history, active community, multiple LLM/vector store integrations, and directly relevant to AI infrastructure hiring managers at top companies.

Positioning: Enterprise-grade agentic architecture · Hybrid local/cloud LLM routing · AWS-native deployment · Privacy-first design

Pre-Work (Before You Write a Single Line of Code)

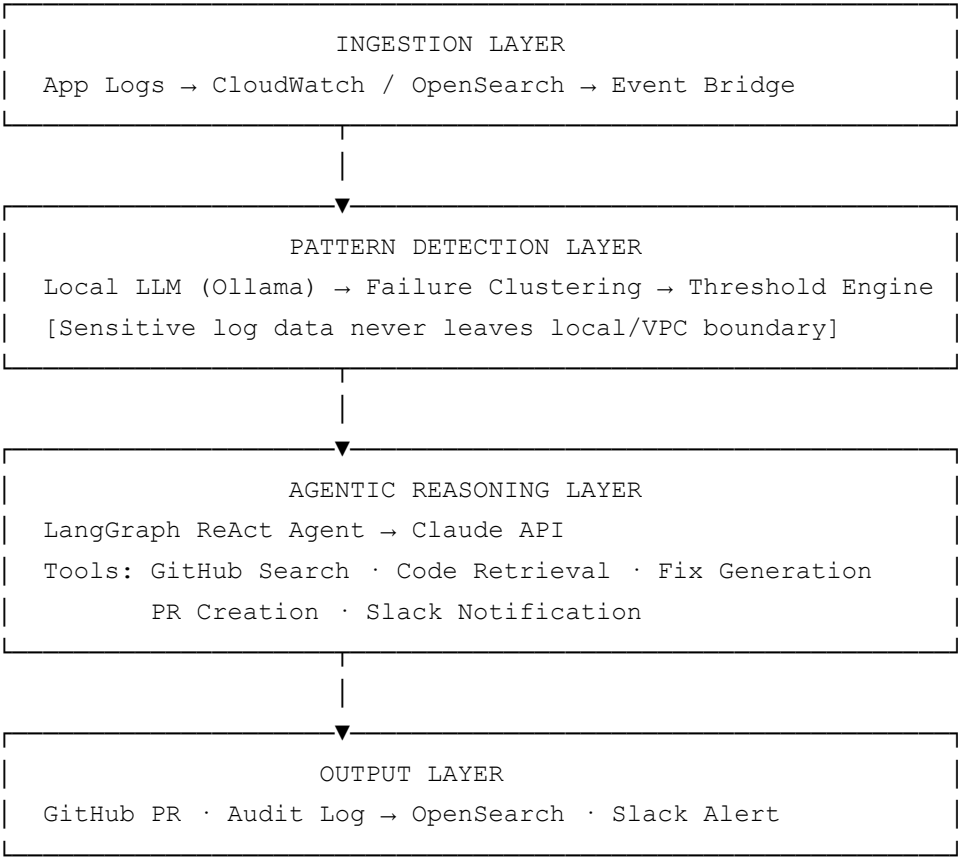
Goal: Understand your test bed deeply so your agent is designed around real failure patterns

- Fork the [Haystack repo](#) and explore the codebase structure
- Browse GitHub Issues — filter by `bug` label, identify 10 recurring failure patterns
- Look at closed PRs to understand how maintainers fix bugs — this shapes your fix generation prompts
- Run Haystack locally, deliberately trigger 3-5 known failures, capture the logs
- Document the failure patterns you'll target — these become your agent's test cases
- Note any sensitive data patterns in logs — informs your local vs Claude routing logic

Time estimate: 1 weekend · This is your most important investment before building

System Architecture

Core Components



Hybrid LLM Routing Strategy

Task	Model	Reason
Log parsing & clustering	Local (Ollama / Llama 3)	Sensitive data, no external calls
Failure pattern summarization	Local	PII redaction before escalation
Root cause reasoning	Claude (API)	Complex multi-step thinking
Code fix generation	Claude (API)	High accuracy needed
PR description writing	Claude (API)	Natural language quality

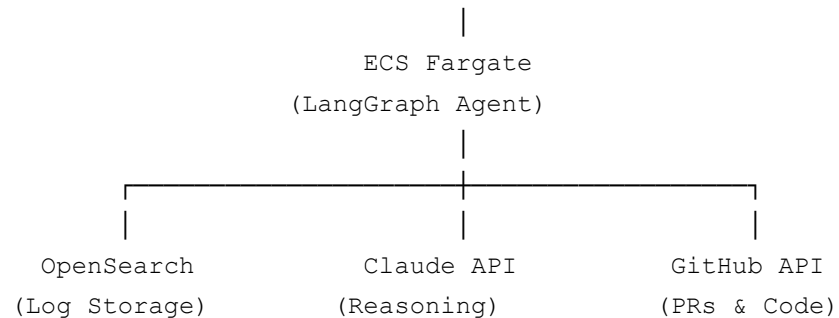
Simple threshold checks

Rule-based

Speed, no LLM needed

AWS Infrastructure

Route: CloudWatch Logs → EventBridge → SQS → Lambda (Ingestion)



- **Local model runtime:** Ollama on EC2 (or on-device for dev)
- **Agent runtime:** ECS Fargate (LangGraph + FastAPI)
- **Storage:** OpenSearch for logs, S3 for code snapshots
- **Secrets:** AWS Secrets Manager for API keys
- **Observability:** Langfuse (self-hosted) for agent trace logging



2.5-Month Execution Plan (Claude Code Accelerated)

Why compressed? Claude Code handles boilerplate, scaffolding, AWS wiring, and Terraform in hours not weeks. You focus on architecture decisions and agent behavior — Claude Code handles implementation.

Weeks 1-2 — Foundation & MVP

Goal: Working end-to-end pipeline

- Use Claude Code to scaffold full project structure — FastAPI, LangGraph, GitHub integration, OpenSearch client
- Implement log ingestion from CloudWatch → OpenSearch
- Build local Ollama-based failure pattern detector
- Wire up basic LangGraph ReAct agent with GitHub tool

- Get first PR auto-created from a simulated failure log
- Write architecture decision records (ADRs)

Claude Code accelerates: Project scaffolding, boilerplate, API wiring **You focus on:** Agent design, tool definitions, failure pattern logic

Deliverable: Demo video of agent detecting a failure and opening a GitHub PR

Weeks 3-4 — Intelligence & Hybrid Routing

Goal: Hybrid LLM routing working cleanly

- Implement routing logic — local model vs Claude based on data sensitivity
- Add PII detection and redaction before Claude API calls
- Improve root cause reasoning with multi-turn Claude prompting
- Add support for multiple repos / monorepo structures
- Implement failure deduplication and recurrence tracking
- Add Slack notification on PR creation

Claude Code accelerates: Routing logic, PII redaction patterns, Slack integration **You focus on:** Routing strategy decisions, prompt engineering, Claude reasoning quality

Deliverable: Hybrid routing demo — same failure routed differently based on log sensitivity

Weeks 5-6 — AWS Deployment & Observability

Goal: Production-grade, deployable on real AWS infrastructure

- Use Claude Code to generate Terraform / CDK for full AWS stack
- Containerize agent with Docker, deploy to ECS Fargate
- Set up EventBridge → SQS → Lambda ingestion pipeline
- Integrate Langfuse for full agent trace observability
- Add CloudWatch dashboards for system health
- Load test with simulated high-frequency failure logs

Claude Code accelerates: Terraform/CDK generation, Dockerfile, Lambda functions **You**

focus on: AWS architecture decisions, security, cost optimization

Deliverable: Live AWS deployment with public Langfuse dashboard showing agent traces

Weeks 7-10 — Polish, Docs & Launch

Goal: Portfolio-ready, open source, publicly visible

- Write comprehensive README with architecture diagrams
- Create a 3-minute demo video walkthrough
- Write first technical blog post: *“Why I built a hybrid local/cloud agentic system and what I learned”*
- Post on LinkedIn with architecture breakdown (3-4 posts)
- Submit to relevant GitHub awesome lists and HN Show HN
- Tag on LinkedIn profile and update resume

Note: Writing, blogging, and visibility building can’t be AI-accelerated — this is your voice and story.

Deliverable: Open source repo with stars, blog post live, LinkedIn visibility

Tech Stack

Layer	Technology
Agent framework	LangGraph + ReAct
Local LLM	Ollama (Llama 3 / Mistral)
Cloud LLM	Claude API (Anthropic)
Backend	FastAPI (Python)
AWS compute	ECS Fargate + Lambda
Log storage	OpenSearch
Infra as code	Terraform / AWS CDK
Observability	Langfuse (self-hosted)

Version control integration	GitHub API (PyGitHub)
Notifications	Slack Webhooks

LinkedIn Narrative (6-month arc)

Month	Post Theme
1	"Why I'm building a hybrid local/cloud agentic system — and the architecture decision behind it"
2	"How I route between local LLMs and Claude based on data sensitivity — a practical pattern"
3	"Deploying a LangGraph agent to AWS ECS — what I learned about production agentic systems"
4	"I open sourced my autonomous code remediation agent. Here's what it does and how it works"

Resume Positioning

Autonomous Code Remediation Agent (*Open Source*) Built a production-grade agentic system on AWS that monitors application logs for recurring failures, traces root causes across GitHub repositories, and autonomously generates fixes via pull requests. Implemented a hybrid LLM routing strategy — local Ollama models for sensitive log processing, Claude API for complex reasoning — deployed on ECS Fargate with full observability via self-hosted Langfuse. *Stack: LangGraph · Claude API · Ollama · FastAPI · AWS ECS · OpenSearch · GitHub API*

Conflict of Interest Checklist

- Review employment agreement IP clause
- Confirm project uses personal time and personal laptop only
- Ensure open source version is generic — no Moody's-specific logic

- Have brief conversation with manager before starting: *"I'm building a general-purpose code remediation agent as a portfolio project on my own time. Does that feel okay?"*
 - Keep Moody's implementation and personal project clearly separated
-

*Built to position for Tech Manager / Principal Engineer roles at AI-first companies · 2026
Timeline compressed from 4 months to 2.5 months using Claude Code for implementation acceleration*